

INT202
Complexity of Algorithms
NP-Completeness

Year 2025-2026

Hard Computational Problems

Some computational problems seem *hard* to solve.

Despite numerous attempts we do not *know* any *efficient* algorithms for these problems.

However, we are also far away from a *proof* that these problems are indeed hard to solve. We simply “don’t know” or are unsure right now.

- **P problems:** decision problems that can be solved in polynomial time ($O(n^k)$ for some constant k).
- **NP problems:** decision problems where a proposed solution can be verified in polynomial time.

Hard Computational Problems (cont.)

In more formal language, we don't know if $NP = P$ or $NP \neq P$.

This is an important question in theoretical computer science! And this lecture is designed to shed some light on this question.

This question is one of the seven Millennium Prizes set out by the Clay Institute, and can earn someone \$1 million dollars for its solution (as well as a place in math/computer science history).

See <https://www.claymath.org/millennium/p-vs-np/> for their description of this problem.

Hard Computational Problems (cont.)

P vs NP Problem

Suppose that you are organizing housing accommodations for a group of four hundred university students. Space is limited and only one hundred of the students will receive places in the dormitory. To complicate matters, the Dean has provided you with a list of pairs of incompatible students, and requested that no pair from this list appear in your final choice.

This is an example of what computer scientists call an NP-problem, since it is easy to check if a given choice of one hundred students proposed by a coworker is satisfactory (i.e., no pair taken from your coworker's list also appears on the list from the Dean's office), however the task of generating such a list from scratch seems to be so hard as to be completely impractical.

Hard Computational Problems (cont.)

P vs NP Problem

Indeed, the total number of ways of choosing one hundred students from the four hundred applicants ($C(400, 100)$, or around 2.4×10^{96}) is greater than the number of atoms in the known universe (10^{82})!

Thus no future civilization could ever hope to build a supercomputer capable of solving the problem by brute force; that is, by checking every possible combination of 100 students.

Hard Computational Problems (cont.)

P vs NP Problem

However, this apparent difficulty may only reflect the lack of ingenuity of programmer. In fact, one of the outstanding problems in computer science is determining whether questions exist whose answer can be quickly checked, but which require an impossibly long time to solve by any direct procedure.

Problems like the one listed above certainly seem to be of this kind, but so far no one has managed to prove that any of them really are so hard as they appear, i.e., that there really is no feasible way to generate an answer with the help of a computer.

Example already encountered

We have already seen one example of these difficult problems where no efficient algorithm (i.e. polynomial-time) is known (i.e. one that will work on *all* inputs in polynomial-time).

This is the $\{0, 1\}$ Knapsack problem.

The $\{0, 1\}$ Knapsack problem

The $\{0, 1\}$ Knapsack problem can be defined in the following way:

$\{0, 1\}$ Knapsack

Input: A collection of items $\{i_1, i_2, \dots, i_n\}$ where item i_j has integer weight $w_j > 0$ and gives integer benefit b_j . Also, a maximum weight W .

Goal: Find a subset of the items whose total weight does not exceed W and that maximizes the total benefit (taking fractional parts of items is *not allowed*).

Note: The dynamic programming algorithm for the $\{0, 1\}$ Knapsack problem runs in time $\theta(nW)$, which is *not* polynomial in the *size* of the problem.

The $\{0, 1\}$ Knapsack problem

The $\{0, 1\}$ Knapsack problem can be defined in the following way:

$\{0, 1\}$ Knapsack

Note: The dynamic programming algorithm for the $\{0, 1\}$ Knapsack problem runs in time $\theta(nW)$, which is *not* polynomial in the *size* of the problem.

$\theta(nW)$ looks like a polynomial time, but it is not, it is pseudo-polynomial.

Time complexity measures the time that an algorithm takes as a function of the length in bits of its input. The dynamic programming solution is indeed linear in the value of W , but exponential in the length of W !

The $\{0, 1\}$ Knapsack problem

The size of input is $\log(W)$ bits.

- In binary representation (base-2), you represent a number using bits (0s and 1s).
- The number 5 is: 101 (3 bits) and $\log_2 5 = 2.32$
- The number 1,048,576 is: 1000000000000000000000 (21 bits) and $\log_2 1048576 = 20$

So, the input size of weight is $j = \log(W)$ (and not mere W).

- $W = 2^j$ (as binary is used).

The final complexity is $O(n * W)$ and this $O(n * W)$ can be rewritten as $O(n * 2^j)$, which exponential in the size of the input.

So, this solution is not polynomial.

Decision/Optimization problems

A *decision problem* is a computational problem for which the output is either *yes* or *no*.

In an *optimization problem* we try to *maximize* or *minimize* some value.

An optimization problem can often be turned into a decision problem by adding a parameter k , and then asking whether the value of the function (from the optimization problem) is *at most* or *at least* k .

If a decision problem is *hard*, then its related optimization version must also be *hard*.

Decision/Optimization problems (cont.)

Example:

Optimization problem: Let G be a connected graph with integer weights on its edges. Find the weight of a minimum spanning tree in G .

Decision problem: Let G be a connected graph with integer weights on its edges, and let k be an integer. Does G have a spanning tree of weight at most k ?

Decision/Optimization problems (cont.)

$\{0, 1\}$ Knapsack (decision version)

Input: A collection of items $\{i_1, i_2, \dots, i_n\}$ where item i_j has integer weight $w_j > 0$ and gives integer benefit $b_j > 0$. Also, a maximum weight W and an integer k .

Question: Is there a subset of the items whose total weight does not exceed W , and whose total benefit is at least k ?

Note that being able to efficiently *answer* decision questions allows us to efficiently *solve* the related optimization problem.

We can use a type of “binary search” to find the optimal solution.

Problems and Languages

Here we start giving a more formal definition of the classes of problems called P and NP.

- The input to a computational problem will be encoded as a finite binary string s . The length of the string is denoted as $|s|$.
- We *identify* a decision problem with a set of strings for which the answer is “yes”.
- We say that the algorithm A for the decision problem *accepts* an input string s if A outputs “yes” on input s .

For some decision problem X , let $L(X)$ denote the set of (binary) strings that should be accepted by an algorithm for that problem. We often refer to $L(X)$ as a *language*.

- We say that an algorithm A *accepts* a language $L(X)$ if A outputs “yes” for each $s \in L(X)$ and outputs “no” otherwise.

Language Examples

Now consider the $\{0, 1\}$ Knapsack Problem (the decision version).

The *encoding* of the problem will be the binary representations of the integer weights, benefits, the maximum weight W , and the parameter k .

The *language* for an algorithm that would solve the $\{0, 1\}$ knapsack problem consists of *all* collections of weight $\{w_i\}$, benefit $\{b_i\}$, total weight W , k parameters for which there is an allowable subset of the given items (i.e. total weight of the selected items is at most W) with total benefit at least k .

Complexity Class P

The *complexity class* P is the set of all decision problems X that can be solved in *worst-case* polynomial time. (Equivalently, P consists of all languages $L(X)$ that can be accepted in polynomial time.)

- That is, they are solvable in $O(p(n))$, where $p(n)$ is a polynomial on n
- A deterministic algorithm is (essentially) one that always computes the correct answer

That is, there is an algorithm A that if $s \in L(X)$, then on input s , A outputs “yes” in time $p(|s|)$, where $|s|$ is the *length* of s , and $p(\cdot)$ is a *polynomial*.

Many, many problems are in the class P. Examples of problems that we have seen in this course include (decision versions of)

- fractional knapsack,
- task scheduling.

Efficient certification

Another important idea when we discuss decision problems is the idea of *certification*.

Example: Sudoku

The problem is given as an $n^2 \times n^2$ array which is divided into blocks of $n \times n$ squares. Some array entries are filled with an integer in the range $[1..n^2]$.

The goal is to complete the array such that each row, column, and block contains each integer from $[1..n^2]$.

Problem

			4
3		2	
	1		3
4			



Solution

1	2	3	4
3	4	2	1
2	1	4	3
4	3	1	2

The Sudoku decision problem is whether a given Sudoku problem has a solution. Finding the solution might be difficult, but verifying the solution is easy.

Efficient certification

Another important idea when we discuss decision problems is the idea of *certification*.

Example: $\{0, 1\}$ Knapsack Problem

Given a collection of weights, benefits, and parameters W and k for an instance of the $\{0, 1\}$ Knapsack Problem, if I propose a subset of these items, it is *easy to check* if those items have total weight at most W and if the total benefit is at least k .

If both of those conditions are satisfied, then we say that the subset of items is a *certificate* for the decision problem, i.e. it *verifies* that the answer to the $\{0, 1\}$ Knapsack decision problem is “yes.”

The idea of *efficient certification* is what is used to define the class of problems called NP.

Complexity class NP (non-deterministic polynomial time)

The class NP consists of all decision problems for which there exists an *efficient certifier*.

An *efficient certifier* for a decision problem X is an algorithm B that takes two input strings s and t . The string s is the input to the decision problem.

“Efficient” means that B is a polynomial-time algorithm, i.e. there is a polynomial function $q(\cdot)$ so that for every string s , we have $s \in L(X)$ if and only if there exists a string t such that $|t| \leq q(|s|)$ and $B(s, t) = \text{“yes”}$.

We can think of the string t as being a “proof” that the answer to the decision problem is “yes” for the input string s .

Certifiers and Certificates

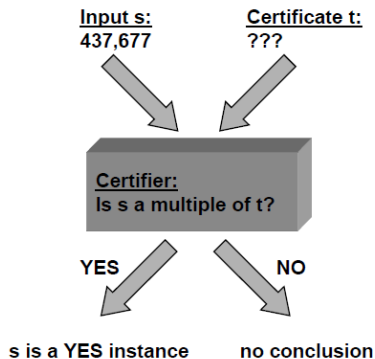
COMPOSITE: Given integer s , is s composite?

Observation. s is composite iff there exists an integer $1 < t < s$ such that s is a multiple of t .

YES instance: $s = 437,669$.

– with certificate $t = 541$ or 809 (a factor)

NO instance: $s = 437,677$.



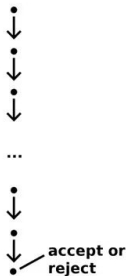
Another (equivalent) definition

An algorithm that *chooses* (by a good guess) some number of *non-deterministic bits* during its execution is called a *non-deterministic algorithm*.

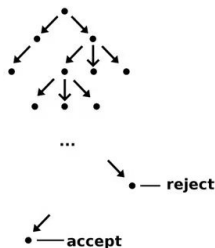
We say that an algorithm *A* *non-deterministically accepts* a string *s* if there exists a *choice of non-deterministic bits* that ultimately leads to the answer “yes.”

Possible exception?
Quantum computers:
no conventional gates.

Deterministic



Non-Deterministic



Complexity Class N P

The class N P is the set of decision problems X (or languages $L(X)$) that can be *non-deterministically accepted* in polynomial time.

(A deterministic computer is what we know)

A nondeterministic computer is one that can “guess” the right answer or solution.

Thus NP can also be thought of as the class of problems whose solutions can be verified in polynomial time.

In the language of the previous definition, for an input string s , the non-deterministic algorithm can generate a (polynomial length) string t , then we use the algorithm B to check if $B(s, t) = \text{“yes.”}$ (And, importantly, if the answer is “yes”, then B will terminate in polynomial time with that output.)

Why N P ?

The search for a string t that will cause an efficient certifier to accept the input s is often viewed as a *non-deterministic search* over a set of possible proofs.

For this reason, N P was named as an acronym for “non-deterministic polynomial time.”

{ 0, 1} Knapsack is in N P

{ 0, 1} Knapsack: Let $I = \{i_1, i_2, \dots, i_n\}$ denote a collection of items where item i_j has integer weight $w_j > 0$ and gives integer benefit b_j . Also, we're given a maximum weight W and an integer k .

Suppose someone proposes a subset of items, it is easy to check:

- if those items have total weight at most W ;
- if the total value is at least k .

If both conditions are true, then the subset of items is a certificate for the decision problem, i.e., it verifies that the answer to the 0/1 knapsack decision problem is "Yes"

P and NP

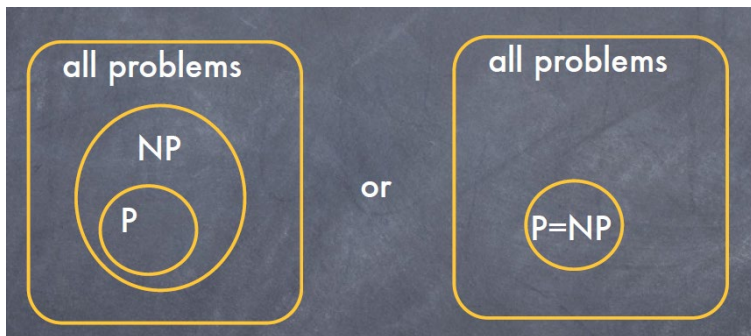
Note that $P \subseteq NP$. Basically, if there is a polynomial-time algorithm A to solve a problem, then we can “ignore” any proposed solution t and return the answer that the algorithm A gives (i.e. whatever A returns, either “yes” or “no”, we give that same answer).

The (million dollar) question that mathematicians and computer scientists *don't know* the answer to is whether $P=NP$ or $P \neq NP$.

There is generally common *belief* that P and NP are different, i.e. there is some problem that is in NP but is not in P .

P and NP

Although poly-time verifiability seems like a weaker condition than poly time solvability, it is unknown whether $P = NP$.



Boolean Circuit

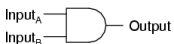
In discrete Math, you learned about *propositional logic*, and the basic operations for combining truth values.

A Boolean Circuit is a directed graph where each vertex, called a *logic gate* corresponds to a simple Boolean function, one of AND, OR, or NOT.

Incoming edges: inputs for its Boolean function

Outgoing edges: outputs

2-input AND gate



A	B	Output
0	0	0
0	1	0
1	0	0
1	1	1

2-input OR gate



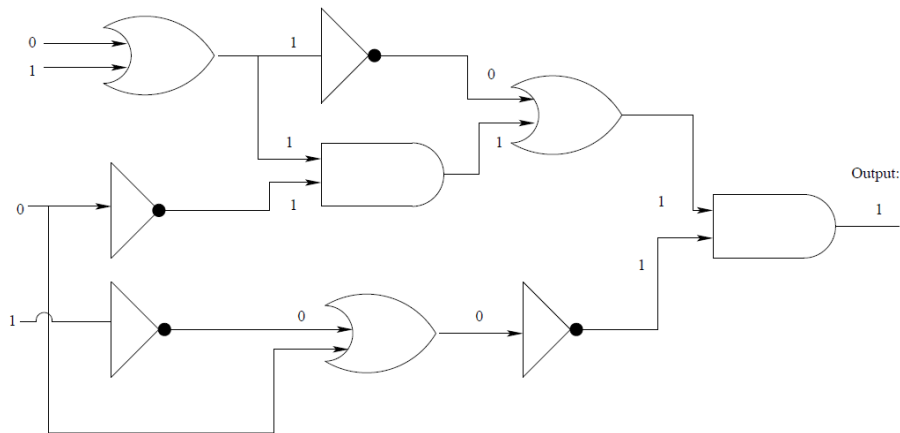
A	B	Output
0	0	0
0	1	1
1	0	1
1	1	1

NOT gate truth table



Input	Output
0	1
1	0

Boolean Circuit - Example

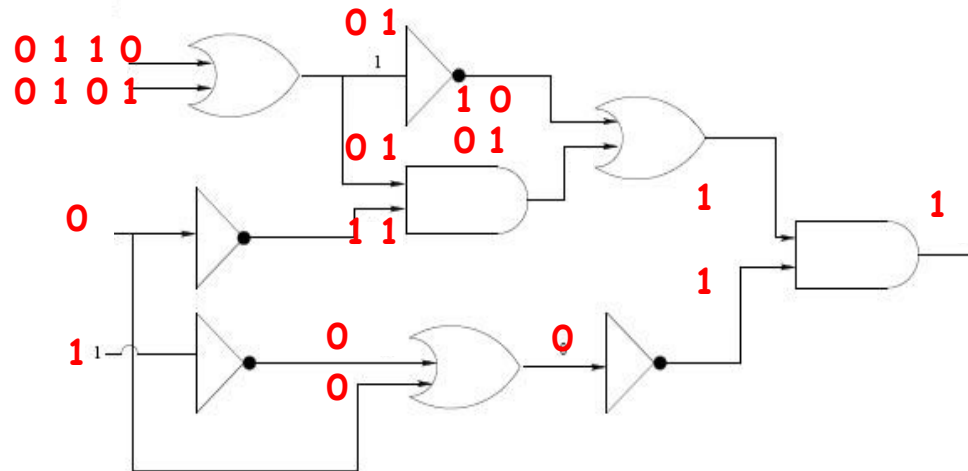


Input: a Boolean Circuit with a single output vertex

Question: is there an assignment of values to the inputs so that the output value is 1?

A non-deterministic algorithm chooses an assignment of input bits and checks deterministically whether this input generates an output of 1.

Circuit-SAT



Circuit-SAT is in NP

Circuit-SAT: Given a Boolean Circuit with a single output node, is there an assignment of values to the inputs so that the output value is 1?

Proof:

Guessing: A non-deterministic algorithm guesses values for all input and the output of each logic gate.

Verification: Verify that each gate's output (we guessed) matches its Boolean function (AND, OR, NOT) based on its inputs. This verification is done in polynomial time with respect to the circuit size.

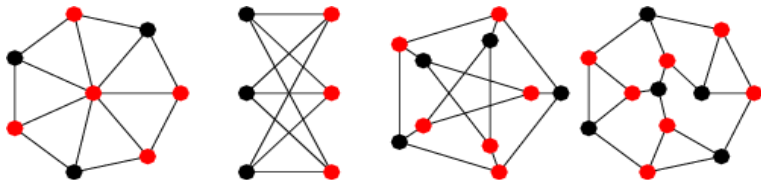
Acceptance Condition: If all gates are consistent with their Boolean functions and the final output is 1, accept the solution.

Since a solution can be verified in polynomial time,
Circuit-SAT $\in$ NP.

Vertex Cover (VC)

Given a graph $G = (V, E)$

A vertex cover is a subset $C \subseteq V$ such that for every edge (v, w) in E , $v \in C$ or $w \in C$



some graphs and their vertex cover (red vertices)

The optimisation problem is to find as small a vertex cover as possible

Vertex Cover is the decision problem that takes a graph G and an integer k and asks whether there is a vertex cover for G containing *at most* k vertices

NP-completeness

Vertex Cover (VC)

Lemma Vertex Cover is in NP

Given an integer k and a graph $G=(V,E)$, does G contain a vertex cover C of size $\leq k$?

Proof:

Guessing: A non-deterministic algorithm guesses a subset $C \subseteq V$ of size at most k . This means we select up to k vertices from the graph G .

Verification: First check that $|C| \leq k$. Then verify that for each edge $(u,v) \in E$, at least one of u or v is in C . This verification is done in polynomial time with respect to the edge size.

Acceptance Condition: If $|C| \leq k$ and all edges pass this check, accept the solution.

Since a solution can be verified in polynomial time, Vertex Cover $\in$ NP.

Polynomial-time reducibility

Another important idea in the theory of NP-completeness is the idea of transforming one problem instance into another problem instance. We formalize this idea below.

- ▶ We say that a language L , defining some decision problem, is *polynomial-time reducible* to a language M , if:
there is a function f , computable in polynomial time, that takes as an input $s \in L$, and transforms it into an input $f(s) \in M$ such that $s \in L$ if and only if $f(s) \in M$.

In other words, we can transform an input for one decision problem into an appropriate input for another decision problem. Furthermore, the first problem has a “yes” solution if and only if the second decision problem has a “yes” solution.

We use notation $L \xrightarrow{\text{poly}} M$ to signify that language L is polynomial time reducible to language M .

N P-hardness

We say that a language M , defining some decision problem, is *NP-hard* if every other language L in NP is polynomial-time reducible to M . So this means that

- ▶ M is NP-hard if for every $L \in \text{NP}$, $L \xrightarrow{\text{poly}} M$.

If a language M is NP-hard and it belongs to NP itself, then M is *NP-complete*.

NP-complete problems are some of the *hardest problems* in NP.

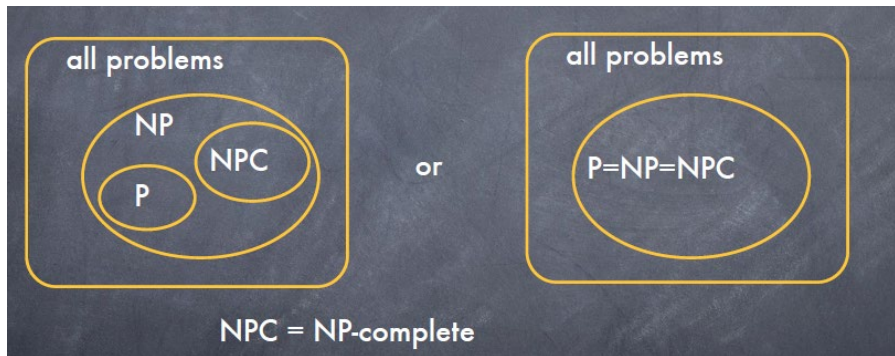
NP-completeness

- **P problems:** decision problems that can be solved in polynomial time ($O(n^k)$ for some constant k).
- **NP problems:** decision problems where a proposed solution can be verified in polynomial time.
- **NP-Complete Problems:** Problems that are:
 - 1) NP-Hard (all NP problems reduce to them) and
 - 2) In NP (solutions can be verified in polynomial time)
- **NP-Hard problems:** Problems that are at least as hard as the hardest NP problems. Every problem in NP can be reduced to them in polynomial time (condition 2 of NPC), but may not be in NP: could be harder or unverifiable.

N P-complete

NP-complete problems is class of "hardest" problems in NP!

- If an NP-complete problem can be solved in polynomial time, then all problems in NP can be solved in polynomial time, and thus $P = NP$.



N P-complete

NP-complete problems is class of "hardest" problems in NP!

- If an NP-complete problem can be solved in polynomial time, then all problems in NP can be solved in polynomial time, and thus $P = NP$.

The question whether $P=NP$ is open since the 70s. It is one of the central open problems in computer science. The question is of theoretical interest as well as of great practical importance.

Pragmatic approach: If your problem is NP-complete, then don't waste time looking for an efficient algorithm, focus on approximations or heuristics.

Cook-Levin Theorem

The Cook-Levin Theorem states that *Circuit-SAT* is N P-complete. [Cook 1971, Levin 1973]

Proof:

1. CIRCUIT-SAT $\in$ NP

Proof: Can verify an input assignment satisfies a circuit by computing the output of a finite number of gates, one of which will be the output of the circuit. This can be done in polynomial time. Thus, by definition of NP, CIRCUIT-SAT $\in$ NP.

2. CIRCUIT-SAT $\in$ NP-Hard

i.e., $L \xrightarrow{\text{poly}}$ CIRCUIT-SAT for every $L \in \text{NP}$

Proof: Complex

Show that any problem in NP can be computed using a Boolean combination circuit (i.e., a computer).

This circuit has a polynomial number of elements and can be constructed in polynomial time. Thus, $L \xrightarrow{\text{poly}}$ CIRCUIT-SAT for all $L \in \text{NP}$.

Thus, CIRCUIT-SAT $\in$ NP-Hard.

N P-complete

We have just noted that there is *at least* one N P-complete problem.

Using polynomial time reducibility we can show existence of other N P-complete problems.

The following useful result also helps to prove N P-complete in many cases.

Lemma: If $L_1 \xrightarrow{\text{poly}} L_2$ and $L_2 \xrightarrow{\text{poly}} L_3$ then $L_1 \xrightarrow{\text{poly}} L_3$

N P-complete

Lemma 13.7: *If $L_1 \xrightarrow{\text{poly}} L_2$ and $L_2 \xrightarrow{\text{poly}} L_3$, then $L_1 \xrightarrow{\text{poly}} L_3$.*

Proof: Since $L_1 \xrightarrow{\text{poly}} L_2$, any instance x for L_1 can be converted in polynomial-time $p(n)$ into an instance $f(x)$ for L_2 , such that $x \in L_1$ if and only if $f(x) \in L_2$, where n is the size of x . Likewise, since $L_2 \xrightarrow{\text{poly}} L_3$, any instance y for L_2 can be converted in polynomial-time $q(m)$ into an instance $g(y)$ for L_3 , such that $y \in L_2$ if and only if $g(y) \in L_3$, where m is the size of y . Combining these two constructions, any instance x for L_1 can be converted in time $q(k)$ into an instance $g(f(x))$ for L_3 , such that $x \in L_1$ if and only if $g(f(x)) \in L_3$, where k is the size of $f(x)$. But, $k \leq p(n)$, since $f(x)$ is constructed in $p(n)$ steps. Thus, $q(k) \leq q(p(n))$. Since the composition of two polynomials always results in another polynomial, this inequality implies that $L_1 \xrightarrow{\text{poly}} L_3$. ■

NP-complete

Suppose that we have a new problem X and we think that X is NP-complete. How can we do this?

- ▶ First, show that $X \in \text{NP}$, i.e. show that X has a polynomial-time nondeterministic algorithm, or equivalently, show that X has an efficient certifier.
- ▶ Secondly, take a *known* NP-complete problem Y , and demonstrate a polynomial time reduction from Y to X , i.e. show that $Y \xrightarrow{\text{poly}} X$.

Conjunctive Normal Form

A Boolean formula is in *Conjunctive Normal Form* (CNF) if it is formed as a collection of *clauses* combined using the operator *AND* ($\wedge$), where each clause is formed by *literals* (variables x_i or their negations $\overline{x_i}$) combined using the operator *OR* ($\vee$).

Some examples are:

$$(\overline{x_1} \vee \overline{x_2} \vee x_4 \vee \overline{x_6}) \wedge (\overline{x_2} \vee x_4 \vee \overline{x_5} \vee x_3)$$

$$(x_2 \vee \overline{x_3} \vee \overline{x_1}) \wedge (x_1 \vee x_3 \vee x_4) \wedge (\overline{x_4} \vee \overline{x_2} \vee \overline{x_5})$$

$$(\overline{x_2} \vee x_1) \wedge (\overline{x_1} \vee x_4 \vee x_3) \wedge (x_4 \vee x_3) \wedge (\overline{x_3} \vee x_2 \vee \overline{x_5} \vee \overline{x_1})$$

CNF-SAT and 3-SAT

CNF-SAT

Input: A Boolean formula in CNF.

Question: Is there an assignment of Boolean values to its variables so that the formula evaluates to 1 (i.e. is the formula *satisfiable*)?

3-SAT

Input: A CNF $C = c_1 \wedge c_2 \wedge \dots \wedge c_m$ of clauses dependent on variables $x_1, x_2, \dots, x_m$ such that each c_i is of the form $x_{i_1} \vee x_{i_2} \vee x_{i_3}$ for $1 \leq i \leq m$.

Question: Is there a truth assignment for variables x_i that satisfies C ?

3-SAT and Vertex Cover (VC)

3-SAT is CNF-SAT in which each clause has exactly three literals.

Theorem: 3-SAT is NP-complete.

VC

Input: An undirected graph G and a positive integer k .

Question: Is there a set C of at most k vertices such that every edge of G is incident to at least one node of C ? (Such a set C is called a vertex cover of G .)

Theorem: VC is NP-complete.

3-Coloring of Graphs

3-COL:

Input: A graph G .

Question: Is G 3-colorable? In other words, is there an assignment of the labels $\{1, 2, 3\}$ to the *vertices* of G so that any two vertices that are adjacent are assigned *different* labels?

3-COL is part of the more general class of problems k -COL, where the vertices of a graph are assigned labels from the set $\{1, 2, \dots, k\}$, and the goal is to assign labels so that adjacent vertices have different labels.

If $k \geq 3$, then k -COL is NP-complete.